

LAPORAN PRAKTIKUM

Pemrograman Berorientasi Objek

Ekspansi Produk Keuangan (Inheritance)

disusun Oleh:

Nama: RIFKI MAULANA

NIM: 2511533007

Kelas: A

DOSEN PENGAMPU: NURFIAH, S.ST, M.Kom

ASISTEN PRAKTIKUM: M. DAWI SYAUQI



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 03 Oktober 2026

A. Langkah Kerja Praktikum

1. Langkah 1: Memastikan Akses Modifier di Rekening.java

Mengubah atribut saldo dan arraylist transaksi menjadi protected supaya yang bisa mengakses hanyalah class itu, class lain dalam package yang sama, dan subclass(baik dari package yang sama maupun dari package yang lain).

```
public class Rekening {  
    // 1. Mengunci atribut dengan 'private'  
    private String nomorRekening;  
    private String namaPemilik;  
    private String pin; // Data sensitif  
  
    // Gunakan protected agar subclass bisa mengaksesnya langsung  
    protected double saldo;  
    protected ArrayList<Transaksi> riwayatTransaksi;  
}
```

2. Langkah 2: Membuat Subclass RekeningTabungan

Disinilah kita menggunakan inheritance, membuat subclass dari class Rekening. Inheritance bisa mengambil atribut dari class parent(jika memungkinkan) maupun method dari class parent.

```
public class RekeningTabungan extends Rekening{  
  
    // Atribut spesifik yang hanya dimiliki oleh Tabungan  
    private double sukuBunga;  
  
    // Constructor Subclass  
    public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal, double sukuBunga) {  
        // super() memanggil constructor kelas induk (Rekening). WAJIB berada di baris pertama!  
        super(nomor, nama, saldoAwal, pinAwal);  
        this.sukuBunga = sukuBunga;  
    }  
  
    public void tambahBungaAkhirBulan() {  
        // Menghitung bunga  
        // mengapa bisa mengakses saldo secara langsung dari class RekeningTabungan?  
        double nominalBunga = saldo * (sukuBunga / 100);  
        saldo += nominalBunga;  
  
        // Mencatat riwayat transaksi  
        String idTrx = "TRX-B-" + System.currentTimeMillis();  
        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));  
  
        System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" + nominalBunga);  
    }  
}
```

- Buat atribut baru di subclass RekeningTabungan yaitu atribut sukuBunga
- Tambahkan method baru tambahBungaAkhirBulan yang berfungsi untuk menambahkan saldo dalam persen sesuai inputan

3. Langkah 3: Membuat Subclass RekeningGiro

Buat RekeningGiro yang berguna untuk limit pinjaman

```
public class RekeningGiro extends Rekening{
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double batasOverdraft )
        // Memanggil inisialisasi dasar dari Superclass
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

    // Getter khusus Giro
    public double getBatasOverdraft() {
        return batasOverdraft;
    }
    // penarikan hingga limit overdraft diselesaikan di modul 5
}
```

- Kita menggunakan getter untuk mengembalikan batasOverdraft

B. Tugas/Latihan praktikum

1. Pembuatan Multi-Produk

Pada file main.java, tepatnya di case 1 buatlah:

- Setelah user menginputkan Saldo awal dan PIN, berikan opsi produk:
"Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis"
- Jika user memilih Tabungan, minta input sukuBunga dalam persen, lalu instansiasi objek RekeningTabungan.
- Jika user memilih Giro, minta input batasOverdraft (limit pinjaman), lalu instansiasi objek RekeningGiro.
- Simpan objek tersebut ke dalam daftarRekening (Upcasting: Subclass otomatis dikenali sebagai tipe Superclass-nya).

```

case 1:
    System.out.print("Masukkan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukkan PIN(6 digit angka): ");
    String pin = input.nextLine();
    System.out.print("Masukkan Saldo Awal: ");
    double saldo = input.nextDouble();
    System.out.println("Pilih Produk(1/2): \n1. Tabungan Umum \n2. Giro Bisnis");
    int pilProduk = input.nextInt();

    if (saldo >= 50000) {
        if (pilProduk == 1) {
            System.out.println("Masukkan suku bunga(dalam persen): ");
            double sukuBunga = input.nextDouble();
            akunAktif = new RekeningTabungan(no, nama, saldo, pin, sukuBunga);
        }
        else if (pilProduk == 2) {
            System.out.println("Masukkan batas overdraft(limit pinjaman): ");
            double batasOverdraft = input.nextDouble();
            akunAktif = new RekeningGiro(no, nama, saldo, pin, batasOverdraft);
        }
        else {
            System.out.println("pilih antara 1 dan 2");
        }
    }
    else {
        System.out.println("saldo tidak boleh kurang dari 50.000");
    }
    break;

```

- Buat input baru berupa pilihan antara angka 1 atau dua untuk Tabungan umum dan giro bisnis, jika tidak program akan menyuruh untuk memasukkan angka 1 atau 2
- Jika input 1 program meminta input sukuBunga, kemudian mengaktifkan akun RekeningTabungan
- Jika input 2 program meminta input baatsOverdraft, kemudian mengaktifkan akun RekeningGiro

2. Simulasi Waktu Akhir Bulan

- a. Tambahkan menu baru: "7. Simulasi Akhir Bulan (Khusus Tabungan)" di CLI.
- b. Di dalam case 7:, periksa apakah akunAktif saat ini adalah objek Tabungan menggunakan operator instanceof
- c. Jika akun tersebut benar RekeningTabungan, lakukan Downcasting ke tipe Tabungan, lalu panggil method tambahBungaAkhirBulan().
- d. Jika bukan, tolak dengan pesan: "Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan."

```

case 7:
    if (akunAktif != null) {
        if (akunAktif instanceof RekeningTabungan) {
            RekeningTabungan rek = (RekeningTabungan) akunAktif;
            rek.tambahBungaAkhirBulan();
        }
        else {
            System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.");
        }
    }
    else {
        System.out.println("Akun belum aktif");
    }
    break;

```

- Program cek dulu apakah akunAktif, kemudian baru men cek apakah akunAktif saat ini menggunakan instanceof, jika yam aka jalankan method tambahBungaAkhirBulan yang ada di subclass RekeningTabungan.
- Jika tidak ada akunAktif maka program menampilkan akun belum aktif
- Jika tidak ada akunAktif untuk RekeningTabungan maka program menyatakan gagal